

Playwright-Notes-10

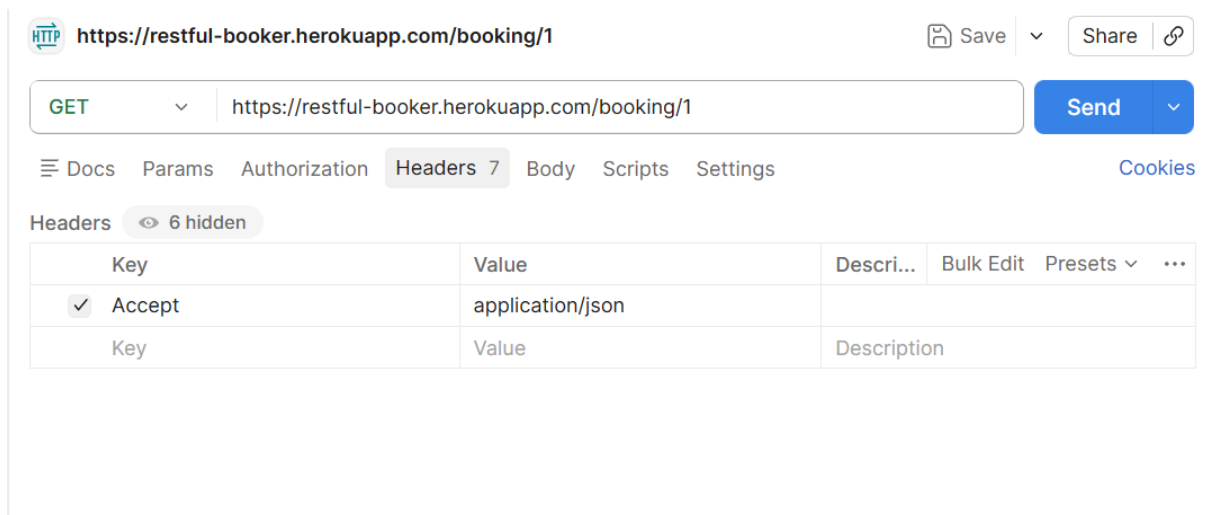
Fetch and Validate API Response headers in Playwright | Playwright Tutorials

Response headers are a vital part of communication in an API (Application Programming Interface). They act like messengers carrying additional information about the response sent back by the server in response to your API request.

- headers() and headersArray() methods can be used to fetch header details of response.

In Postman:

- Refer to the below image-click on the '+' icon in the top left corner, click on new and then HTTP and then set the parameters according to the below image:



In VSCode:

- In the tests folder create a file called `APITestingResponseVerification.spec.ts` and include the below code:

```
import { test, expect } from "@playwright/test";

test("Fetch and Validate Response Header", async ({ request }) => {
  const getResponse = await request.get("/booking/1")
  const headersValue = getResponse.headers();
  console.log(headersValue);
  expect(headersValue.server).toEqual("Cowboy");
  expect(headersValue["x-powered-by"]).toEqual("Express");
});
```

```

console.log("*****")

const headersArrayValues = getResponse.headersArray();
console.log(headersArrayValues);
expect(headersArrayValues.length).toBe(11);
headersArrayValues.forEach((header) => {
    console.log(header.name + "://" + header.value)
})
})

```

```

test("Fetch and Validate Response Header", async ({ request }) => {
    const getResponse = await request.get("/booking/1")
    const headersValue = getResponse.headers();
    console.log(headersValue);
    expect(headersValue.server).toEqual("Heroku");
    expect(headersValue["x-powered-by"]).toEqual("Express");

    console.log("*****")

    const headersArrayValues = getResponse.headersArray();
    console.log(headersArrayValues);
    expect(headersArrayValues.length).toBe(10);
    headersArrayValues.forEach((header) => {
        console.log(header.name + "://" + header.value)
    })
})

```

Headers

200 OK • 276 ms • 894 B

Key	Value
Content-Length	142
Content-Type	application/json; charset=utf-8
Date	Wed, 27 May 2026 23:59:53 GMT
Etag	W/"8e-9c0bhPdJjxEwJD8crnA2TaHdnHE"
Nel	{"report_to":"heroku-nel","response_headers":["V"
Report-To	{"group":"heroku-nel","endpoints":[{"url":"https://"
Reporting-Endpoints	heroku-nel="https://nel.heroku.com/reports?s=s

Note: the 10 and Heroku parameters in the above code are set from the image- count the number of headers in the below terminal and refer to the server name which is “Heroku”

Explanation of code:

Playwright

Response

Headers

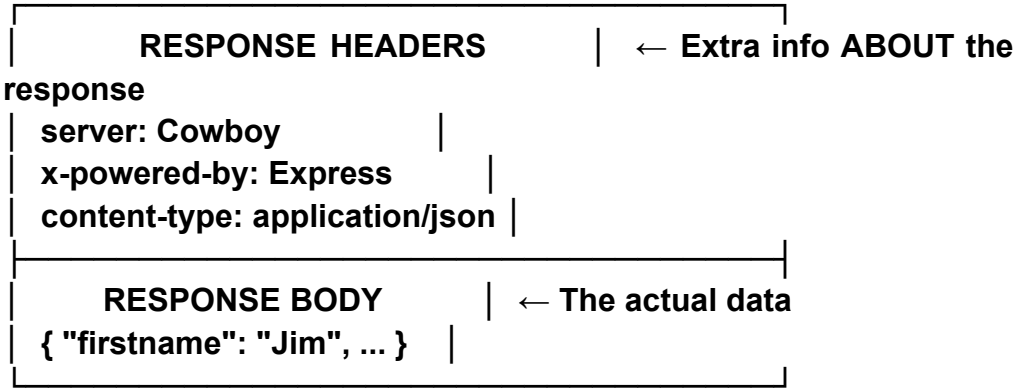
Test

Explanation

This test is different from all previous ones — instead of checking the response body, it checks the response headers that come back from the server.

What Are Response Headers?

Every API response has two parts:



Headers are like the envelope and body is like the letter inside

Part 1 — The GET Request

```
const getResponse = await request.get("/booking/1")
```

- Sends a simple GET request to fetch booking #1
 - No auth, no body needed — just fetching
-

Part 2 — `.headers()` Method

```
const headersValue = getResponse.headers();  
console.log(headersValue);
```

- `.headers()` returns all headers as a flat key-value object
- Console output looks like this:

```
{  
  "server": "Cowboy",  
  "x-powered-by": "Express",  
  "content-type": "application/json",  
  "date": "Mon, 27 May 2026 10:00:00 GMT",  
  ...  
}
```

```
expect(headersValue.server).toEqual("Cowboy");
```

- Checks the server header equals `"Cowboy"`
 - `"Cowboy"` is the name of the web server software running the API
-

```
expect(headersValue["x-powered-by"]).toEqual("Express");
```

- Checks the x-powered-by header equals `"Express"`
- Uses bracket notation `[ "x-powered-by" ]` instead of dot notation because the header name contains a hyphen -

- **Express** is the Node.js framework powering the backend
-

Why Bracket vs Dot Notation?

//  Works — no special characters in name
headersValue.server

//  Fails — hyphen breaks dot notation
headersValue.x-powered-by

//  Works — bracket notation handles special characters
headersValue["x-powered-by"]

Part 3 — **.headersArray()** Method

```
const headersArrayValues = getResponse.headersArray();  
console.log(headersArrayValues);
```

- **.headersArray()** returns headers as an array of objects instead of a flat object
- Each item has a **name** and **value** property
- Output looks like this:

```
[  
  { "name": "server",    "value": "Cowboy"      },  
  { "name": "x-powered-by","value": "Express"    },  
  { "name": "content-type","value": "application/json" },  
  { "name": "date",     "value": "Mon, 27 May 2026" },  
  ...  
]
```

```
expect(headersArrayValues.length).toBe(11);
```

- Verifies there are exactly 11 headers in the response
 - Useful for detecting if the server accidentally adds or removes headers
-

```
headersArrayValues.forEach((header)=>{
```

```
    console.log(header.name + "::" + header.value)
  })
```

- Loops through every header and prints it in **name::value** format
- Console output looks like:

```
server::Cowboy
x-powered-by::Express
content-type::application/json
date::Mon, 27 May 2026 10:00:00 GMT
...
```

.headers() Comparison

vs

.headersArray()

.headers()
Flat
object { }

.headersArray()
Array of objects []

headers
Value.s
erver

header.name,
header.value

Checking
specific
headers

Looping through all
headers

✗ Not
easy

✓ Use `.length`

```
{  
  server:  
  "Cowboy"  
}
```

```
[{      name:  
  "server", value:  
  "Cowboy" }]
```

Complete Flow Diagram

STEP 1: GET /booking/1

→ Fetch booking from server

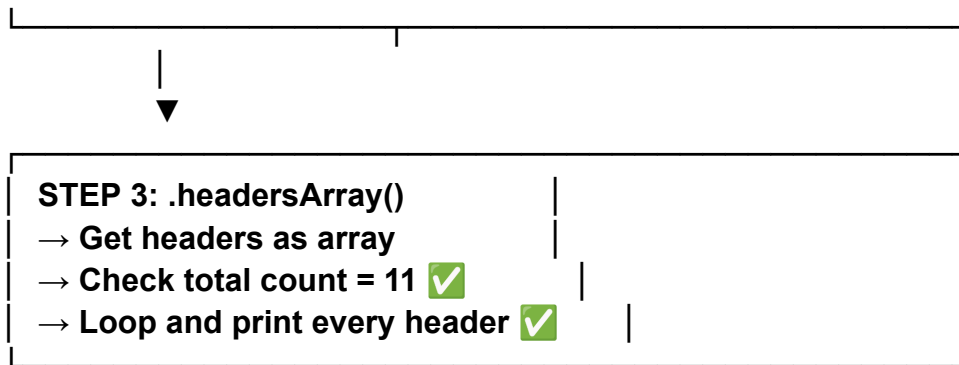


STEP 2: `.headers()`

→ Get headers as flat object

→ Check `server = "Cowboy"` ✓

→ Check `x-powered-by = "Express"` ✓



Layman's Terms

Think of an API response like a package delivered to your door:

- **Response Body** — The actual item inside the box (booking data)
- **Response Headers** — The label on the outside of the box (who sent it, what's inside, how big it is, delivery date)

The two methods are like reading the label differently:

- **`.headers()`** — Like reading a summary label — you can quickly find any specific detail: *"Who delivered this?"* → **"Cowboy"**, *"What courier used?"* → **"Express"**
- **`.headersArray()`** — Like reading every line on the label one by one — you go through each piece of info systematically and can count how many lines there are (11 total)

Why test headers at all?

- To confirm the right server is responding (not a fake/wrong server)
- To verify the right technology is being used (**Express**)
- To detect unexpected changes to the server configuration — if suddenly there are 12 headers instead of 11, something changed!

 **Real world use** — In production testing, header checks catch security issues (missing security headers), configuration problems (wrong server responding), and API version mismatches — things the response body alone would never reveal!

Generate and Use Basic Authentication Token for API Testing with Playwright | Playwright Tutorials

Authentication in APIs is a critical aspect of security, ensuring that only authorized users can access or manipulate the resources provided by the API.

There are several common methods for API authentication, like API Keys, Basic Authentication, OAuth, OAuth 2.0, JWT, Bearer token , etc.

<https://www.youtube.com/watch?v=IZh2t8UAnjk&list=PL83cimSRP5ZmwhC6u255huRwSi9tIP-nc&index=54>

- Refer to the above link of the Postman implementation

In VSCode:

- Create a file under the tests folder called **APITestingAuthentication.spec.ts** and include the below code:

```
import { expect, test } from "@playwright/test"

let tokenValue: string;
test.beforeAll("Basic Auth", async ({ request }) => {
    const respToken = await
request.post("https://restful-booker.herokuapp.com/auth", {
    data: {
        "username": "admin",
        "password": "password123"
    }
    })
    tokenValue = (await respToken.json()).token;
})

test("Authentication of Put Call using basic
auth", async ({ request }) => {
    const respPut = await
request.put("https://restful-booker.herokuapp.com/booking/1"
,{
    headers:{
        Cookie:"token=" + tokenValue
    },
    data:{
        "firstname" : "WishInfinite",
        "lastname" : "Brown",
        "totalprice" : 111,
        "depositpaid" : true,
```

```

        "bookingdates" : {
            "checkin" : "2018-01-01",
            "checkout" : "2019-01-01"
        },
        "additionalneeds" : "Breakfast"
    }
});

expect(respPut.status()).toBe(200);
})

test("Authentication of Delete Call using basic
auth", async ({request}) => {
    const respDelete = await
request.delete("https://restful-booker.herokuapp.com/booking
/1", {
    headers: {
        Cookie: "token=" + tokenValue
    }
    })
    expect(respDelete.status()).toBe(201);
})

// test("Authentication with API Key", async ({request}) => {
//
//     const resp = await
request.put("https://restful-booker.herokuapp.com/booking/1"
, {
//     headers: {
//         Authorization: "Basic
YWRtaW46cGFzc3dvcmQxMjM="
//     },
//     data: {
//         "firstname" : "WishInfinite",
//         "lastname" : "Brown",
//         "totalprice" : 111,
//         "depositpaid" : true,
//         "bookingdates" : {
//             "checkin" : "2018-01-01",
//             "checkout" : "2019-01-01"
//         },
//         "additionalneeds" : "Breakfast"
//     }
//     })

```

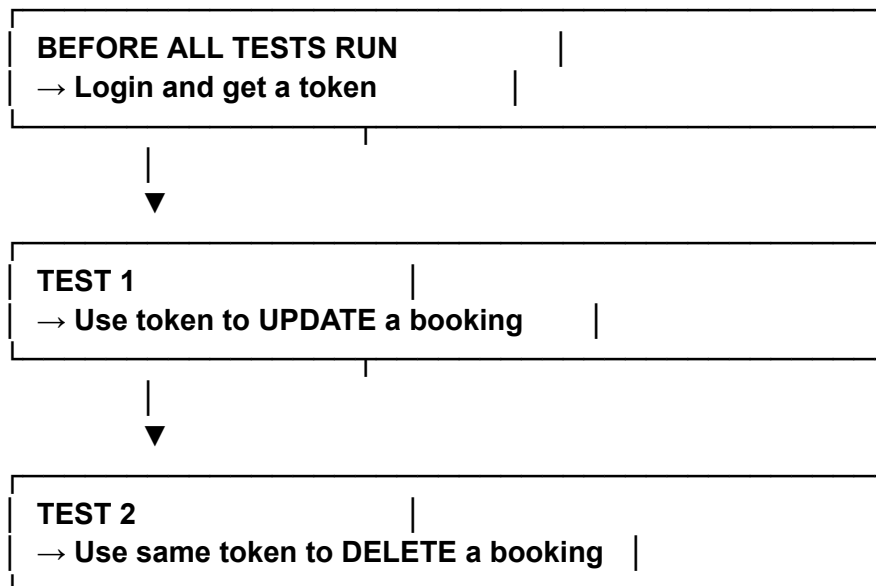
```
//     expect(resp.status()).toBe(200);  
// })
```

Explanation of code:

Playwright Authentication Test Explanation

This test introduces a real-world authentication flow — first getting a token, then using it across multiple tests. This is how most real APIs work.

The Big Picture Flow



Part 1 — The Shared Variable

```
let tokenValue: string;
```

- Declares a shared variable outside all tests
 - Starts empty — gets filled in **beforeAll**
 - Both tests can access and use this variable
 - Like a locker that all tests share
-

Part 2 — **test.beforeAll** (The Login Step)

```
test.beforeAll("Basic Auth", async ({ request }) => {  
    const respToken = await  
    request.post("https://restful-booker.herokuapp.com/auth", {  
        data: {  
            "username": "admin",  
            "password": "password123"  
        }  
    })  
    tokenValue = (await respToken.json()).token;  
})
```

Breaking it down:

```
test.beforeAll(...)
```

- Runs once before ALL tests in this file
- Perfect for login/setup that only needs to happen once

```
await request.post("https://restful-booker.herokuapp.com/auth", {  
    data: {  
        "username": "admin",  
        "password": "password123"  
    }  
})
```

- Sends credentials to the auth endpoint
- Like logging into a website with username and password
- Server responds with a token if credentials are correct

```
tokenValue = (await respToken.json()).token;
```

- Extracts the token from the response and stores it in **tokenValue**
- Response from server looks like:

```
{  
  "token": "abc123xyz789" ← This gets stored in tokenValue  
}
```

Part 3 — Test 1: PUT Using Token

```
test("Authentication of Put Call using basic auth", async({request})=>{
    const respPut = await
    request.put("https://restful-booker.herokuapp.com/booking/1",{
        headers:{
            Cookie:"token=" + tokenValue // ← Token attached here
        },
        data:{
            "firstname" : "WishInfinite",
            ...
        }
    });
    expect(respPut.status()).toBe(200);
})

Cookie:"token=" + tokenValue
```

- Attaches the token as a Cookie header
 - If `tokenValue = "abc123"` then this becomes `Cookie: "token=abc123"`
 - Server reads this cookie and verifies your identity
 - Without it → server returns 403 Forbidden
-

Part 4 — Test 2: DELETE Using Same Token

```
test("Authentication of Delete Call using basic auth",
    async({request})=>{
        const respDelete = await
        request.delete("https://restful-booker.herokuapp.com/booking/1",{
            headers:{
                Cookie:"token=" + tokenValue // ← Same token reused
            }
        })
        expect(respDelete.status()).toBe(201);
    })
```

- Reuses the exact same token from `beforeAll`
 - No data body needed — just the URL + token
 - Server returns 201 on successful delete (quirk of this API)
-

Part 5 — The Commented Out Test

```
// test("Authentication with API Key",async({request})=>{
//   const resp = await request.put("...",{
//     headers:{
//       Authorization:"Basic YWRtaW46cGFzc3dvcmQxMjM="
//     },
//     ...
//   })
// })
```

- This test is disabled using `//` comments
- Shows an alternative auth method — Basic Auth with API Key
- Instead of a dynamic token it uses a hardcoded Base64 key
- Kept in code for reference or future use

3 Authentication Methods Compared

	How It Works	Verdict	Security
1		1	
2		2	
3		3	
4		4	
5		5	
6		6	
7		7	
8		8	
9		9	
10		10	
11		11	
12		12	
13		13	
14		14	
15		15	
16		16	
17		17	
18		18	
19		19	
20		20	
21		21	
22		22	
23		23	
24		24	
25		25	
26		26	
27		27	
28		28	
29		29	
30		30	
31		31	
32		32	
33		33	
34		34	
35		35	
36		36	
37		37	
38		38	
39		39	
40		40	
41		41	
42		42	
43		43	
44		44	
45		45	
46		46	
47		47	
48		48	
49		49	
50		50	
51		51	
52		52	
53		53	
54		54	
55		55	
56		56	
57		57	
58		58	
59		59	
60		60	
61		61	
62		62	
63		63	
64		64	
65		65	
66		66	
67		67	
68		68	
69		69	
70		70	
71		71	
72		72	
73		73	
74		74	
75		75	
76		76	
77		77	
78		78	
79		79	
80		80	
81		81	
82		82	
83		83	
84		84	
85		85	
86		86	
87		87	
88		88	
89		89	
90		90	
91		91	
92		92	
93		93	
94		94	
95		95	
96		96	
97		97	
98		98	
99		99	
100		100	
101		101	
102		102	
103		103	
104		104	
105		105	
106		106	
107		107	
108		108	
109		109	
110		110	
111		111	
112		112	
113		113	
114		114	
115		115	
116		116	
117		117	
118		118	
119		119	
120		120	
121		121	
122		122	
123		123	
124		124	
125		125	
126		126	
127		127	
128		128	
129		129	
130		130	
131		131	
132		132	
133		133	
134		134	
135		135	
136		136	
137		137	
138		138	
139		139	
140		140	
141		141	
142		142	
143		143	
144		144	
145		145	
146		146	
147		147	
148		148	
149		149	
150		150	
151		151	
152		152	
153		153	
154		154	
155		155	
156		156	
157		157	
158		158	
159		159	
160		160	
161		161	
162		162	
163		163	
164		164	
165		165	
166		166	
167		167	
168		168	
169		169	
170		170	
171		171	
172		172	
173		173	
174		174	
175		175	
176		176	
177		177	
178		178	
179		179	
180		180	
181		181	
182		182	
183		183	
184		184	
185		185	
186		186	
187		187	
188		188	
189		189	
190		190	
191		191	
192		192	
193		193	
194		194	
195		195	
196		196	
197		197	
198		198	
199		199	
200		200	
201		201	
202		202	
203		203	
204		204	
205		205	
206		206	
207		207	
208		208	
209		209	
210		210	
211		211	
212		212	
213		213	
214		214	
215		215	
216		216	
217		217	
218		218	
219		219	
220		220	
221		221	
222		222	
223		223	
224		224	
225		225	
226		226	
227		227	
228		228	
229		229	
230		230	
231		231	
232		232	
233		233	
234		234	
235		235	
236		236	
237		237	
238		238	
239		239	
240		240	
241		241	
242		242	
243		243	
244		244	
245		245	
246		246	
247		247	
248		248	
249		249	
250		250	
251		251	
252		252	
253		253	
254		254	
255		255	
256		256	
257		257	
258		258	
259		259	
260		260	
261		261	
262		262	
263		263	
264		264	
265		265	
266		266	
267		267	
268		268	
269		269	
270		270	
271		271	
272		272	
273		273	
274		274	
275		275	
276		276	
277		277	
278		278	
279		279	
280		280	
281		281	
282		282	
283		283	
284		284	
285		285	
286		286	
287		287	
288		288	
289		289	
290		290	
291		291	
292		292	
293		293	
294		294	
295		295	
296		296	
297		297	
298		298	
299		299	
300		300	
301		301	
302		302	
303		303	
304		304	
305		305	
306		306	
307		307	
308		308	
309		309	
310		310	
311		311	
312		312	
313		313	
314		314	
315		315	
316		316	
317		317	
318		318	
319		319	
320		320	
321		321	
322		322	
323		323	
324		324	
325		325	
326		326	
327		327	
328		328	
329		329	
330		330	
331		331	
332		332	
333		333	
334		334	
335		335	
336		336	
337		337	
338		338	
339		339	
340		340	
341		341	
342		342	
343		343	
344		344	
345		345	
346		346	
347		347	
348		348	
349		349	
350		350	
351		351	
352		352	
353		353	
354		354	
355		355	
356		356	
357		357	
358		358	
359		359	
360		360	
361		361	
362		362	
363		363	
364		364	
365		365	
366		366	
367		367	
368		368	
369		369	
370		370	
371		371	
372		372	
373		373	
374		374	
375		375	
376		376	
377		377	
378		378	
379		379	
380		380	
381		381	
382		382	
383		383	
384		384	
385		385	
386		386	
387		387	
388		388	
389		389	
390		390	
391		391	
392		392	
393		393	
394		394	
395		395	
396		396	
397		397	
398		398	
399		399	
400		400	
401		401	
402		402	
403		403	
404		404	
405		405	
406		406	
407		407	
408		408	
409		409	
41			

← Returns 201



Layman's Terms

Think of it like visiting a theme park:

- **test.beforeAll** — You go to the entrance gate first and show your ID (**admin/password123**). The gate gives you a wristband (token) that proves you paid and are allowed in
- **tokenValue** — The wristband you wear all day — you only get it once at the entrance but use it everywhere inside
- Test 1 (PUT) — You go to a ride and show your wristband — the ride operator sees it and lets you on (updates the booking)
- Test 2 (DELETE) — You go to another attraction and show the same wristband — same token, no need to go back to the entrance again
- Commented test — Like an alternative entry method — some parks let season pass holders use a different gate. It works too but isn't being used right now, just kept as a backup option

🔑 The key benefit of **beforeAll** — You only log in once instead of logging in before every single test. This saves time and mirrors how real users behave — you log in once and your session works across the whole website until it expires!

Execute Test in Multiple Environments with Playwright | Handle Multiple Environments | Playwright Tutorials

Handling multiple environments with Playwright lets you run tests on different setups like development, staging, UAT, production etc.

- Mainly we would require the Dotenv package to handle multiple Environments.
- We would also be using Cross-env package, The cross-env package for Node.js helps manage environment variables across platforms and operating systems. It ensures (more or less) consistent behaviour across different platforms by setting environment variables regardless of the operating system

Batch Terminal refers to the command prompt (cmd.exe) provided by Microsoft Windows.

PowerShell Terminal is a command-line interface provided by Microsoft.

Bash Terminal is the default terminal on most Unix-based operating systems, including Linux and macOS.'

Commands to execute test with Specific environment on different terminals -

Batch (cmd) Terminal -

```
set <var_name> = <value>
npx playwright test
```

PowerShell Terminal -

```
$env:<var_name>='<value>';
npx playwright test
```

Bash Terminal -

```
<var_name>=value npx playwright test
```

PowerShell Terminal

```
$env:<var_name>='<value>';
npx playwright test
```

What It Means

The variable name (e.g. URL, USERNAME1, PASSWORD)

The actual value (e.g. <https://saucedemo.com>, admin, secret_sauce)

Real Example:

```
$env:URL='https://www.saucedemo.com';  
npx playwright test
```

Bash Terminal (Mac/Linux)

```
<var_name>=value npx playwright test
```

What It Means

The variable name (e.g. **URL**, **USERNAME1**, **PASSWORD**)

The actual value (e.g. **https://saucedemo.com**, **admin**, **secret_sauce**)

Real Example:

```
URL=https://www.saucedemo.com npx playwright test
```

Key Differences Between The Two

PowerShell

`$env:` required

Single quotes `' '` required

Semicolon `;` between
variable and command

```
$env:URL='https://site.com';
```

Bash

Nothing
— just
the name

No
quotes
needed

Just a
space

```
URL=https://site.com
```



Layman's Terms

Think of it like writing your name on your lunchbox before putting it in the fridge:

- Variable name (`var_name`) — The label on the box — e.g. `URL` or `PASSWORD`
- Value — The actual content inside — e.g. `https://saucedemo.com` or `secret_sauce`

- PowerShell needs `$env:` first — like the fridge requires a special sticker format
- Bash is simpler — just write `NAME=value` directly — like writing straight on the box with a marker

UAT-User Acceptance Testing

UAT — User Acceptance Testing

UAT (User Acceptance Testing) is the final phase of software testing where real end-users or clients test the software to verify it meets their business requirements and works as expected in real-world scenarios — before it goes live.

The core idea:

"Does this software actually do what the business/client asked for?"

It is not about finding bugs (that's done earlier) — it's about validating that the right thing was built.

Where UAT fits in the testing lifecycle:

Unit Testing → Integration Testing → System Testing → UAT → Production

UAT is the last gate before release.

Who performs UAT?

- Actual end users
- Clients or business stakeholders
- Product owners
- Sometimes a dedicated QA team acting as the user

Not the developers who built it.

Types of UAT:

- Alpha Testing — done internally by the client/user at the developer's site

- Beta Testing — done externally by real users in their own environment
 - Contract UAT — verifies the software meets contractual requirements
 - Regulation UAT — checks compliance with legal/regulatory rules
-

Simple real-world example:

A bank hires a company to build a loan application portal. After all technical testing is done, actual bank employees log in and process mock loan applications to confirm everything works the way their business process requires — that is UAT.

UAT Pass/Fail:

- Pass → software goes live
- Fail → issues are reported back to developers for fixes before release

<https://www.youtube.com/watch?v=c-TC9QEuMuo&list=PL83cimSRP5ZmwhC6u255huRwSi9tIP-nc&index=55>

- Refer to the above link for installation and terminal commands and what configurations to add to the package.json file

In VSCode:

- Create a file under the tests folder called `HandleMultipleEnv.spec.ts` and include the below code:

```
import {test} from "@playwright/test"

test("Handle Multiple
Environemnts", async ({page}) =>{
    console.log(process.env.URL)
    console.log(process.env.USERNAME1)
    console.log(process.env.PASSWORD)
    //const urlVal = <string> process.env.URL;
    const urlVal = process.env.URL as string;
    await page.goto(urlVal)

    await
page.locator('[data-test="username"]').fill(<str
ing> process.env.USERNAME1)

    await
page.locator('[data-test="password"]').fill(proc
ess.env.PASSWORD as string)
```

```
await page.waitForTimeout(7000)
}))
```

Explanation of the code:

Playwright Environment Variables Test Explanation

This test introduces environment variables — a way to handle different settings (URLs, usernames, passwords) without hardcoding them directly in the test code.

What Are Environment Variables?

Instead of this (hardcoded) ❌:

```
await page.goto("https://www.saucedemo.com")
```

You do this (environment variable) ✅:

```
await page.goto(process.env.URL)
```

- Environment variables are stored outside your code
 - They can be different per environment (dev, staging, production)
 - They protect sensitive data like passwords from being visible in code
-

Part 1 — Reading Environment Variables

```
console.log(process.env.URL)
console.log(process.env.USERNAME1)
console.log(process.env.PASSWORD)
```

- `process.env` is Node.js's way of reading environment variables
- These three lines just print the values to the console for debugging
- Output might look like:

```
https://www.saucedemo.com
standard_user
secret_sauce
```

Part 2 — Type Casting

// Option 1 — Angle bracket style

```
const urlVal = <string> process.env.URL;
```

// Option 2 — 'as' keyword style (used in this code)

```
const urlVal = process.env.URL as string;
```

- Both do exactly the same thing
- `process.env.URL` returns `string | undefined` — it might not exist
- Adding `as string` tells TypeScript: "trust me, this value exists and is a string"
- Without it TypeScript throws an error because the value could be undefined

Part 3 — Using Variables in Actions

```
await page.goto(urlVal)
```

- Navigates to whatever URL is stored in `process.env.URL`
- Could be dev, staging, or production depending on what's set

```
await page.locator('[data-test="username"]').fill(<string>  
process.env.USERNAME1)
```

- Finds the username input field on the page
- Fills it with the value from `process.env.USERNAME1`

- Note: `USERNAME1` not `USERNAME` — likely because `USERNAME` is a reserved system variable on some machines

```
await
page.locator('[data-test="password"]').fill(process.env.PASSWORD as string)
```

- Finds the password input field
- Fills it with the value from `process.env.PASSWORD`
- Uses `as string` casting — same as the URL variable

```
await page.waitForTimeout(7000)
```

- Waits 7 seconds after filling the form
- Used here to visually observe what happened before the test ends
- ⚠ In real tests `waitForTimeout` is bad practice — use proper wait conditions instead

Where Do Environment Variables Come From?

Option 1 — `.env` File

```
# .env file in your project
URL=https://www.saucedemo.com
USERNAME1=standard_user
PASSWORD=secret_sauce
```

Option 2 — Terminal Command

```
# Mac/Linux
URL=https://www.saucedemo.com
USERNAME1=standard_user PASSWORD=secret_sauce
npx playwright test
```

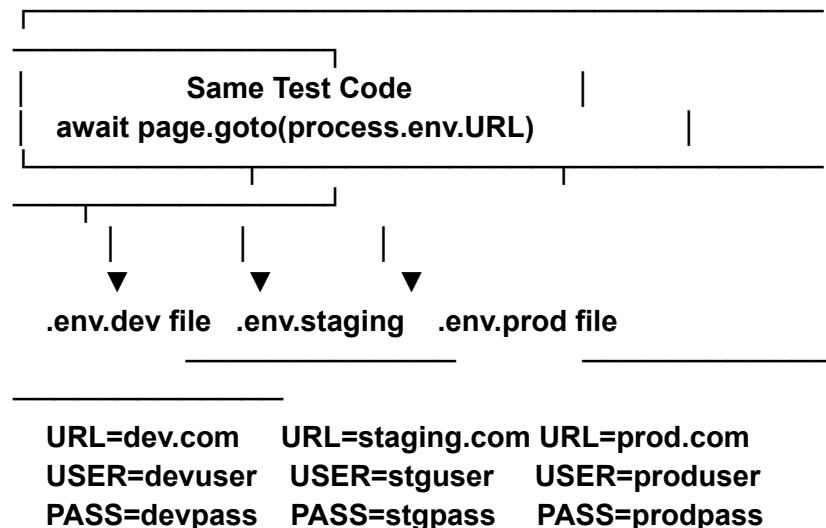
```
# Windows
set URL=https://www.saucedemo.com && npx playwright
test
```

Option 3 — playwright.config.ts

```
import { defineConfig } from "@playwright/test";
import dotenv from "dotenv";
dotenv.config(); // ← Loads .env file automatically
```

```
export default defineConfig({
  use: {
    baseURL: process.env.URL
  }
});
```

How Multiple Environments Work



- One test file works across all environments
 - Just swap the **.env** file or set different variables
-

Hardcoded vs Environment Variables Comparison

Layman's Terms

Think of it like a universal remote control:

- **Hardcoded values** — Like a remote that only works with one specific TV — if you change the TV, the remote is useless and you have to buy a new one
- **Environment variables** — Like a universal remote that can be programmed for any TV — the buttons (test code) stay the same, you just change the settings for which TV (environment) you want to control

Real world analogy:

- **`process.env.URL`** — Like a sticky note on your desk that says which office location to go to today — the sticky note can say "New York" on Monday and "Chicago" on Tuesday, but your commute routine stays the same
- **`process.env.USERNAME1`** — Your staff ID for that specific location

- `process.env.PASSWORD` — Your door access code for that building
- `as string` — Like telling your GPS *"trust me, this address exists"* when it shows a warning

🔑 The golden rule — NEVER put passwords or sensitive data directly in your code! If your code goes to GitHub, the whole world can see it. Environment variables keep secrets secret — the `.env` file stays on your machine and never gets uploaded!

- Then create the `.env.dev` and `.env.uat` in the `env-files` folder which will be created in the project directory
- In the `.env.dev` file include the below code:

```
URL="https://www.saucedemo.com/"
USERNAME1="standard_user"
PASSWORD="secret_sauce"
```

- In the `.env.uat` file include the below code:

```
URL="https://www.saucedemouat.com/"
USERNAME1="standard_user1"
PASSWORD="secret_sauce1"
```

- In the `playwright.config.ts` file include the following code especially the `dotenv` configurations:

```
import { defineConfig, devices } from '@playwright/test';
import dotenv from "dotenv"
import path from 'path';

/**
 * Read environment variables from file.
 * https://github.com/motdotla/dotenv
 */
// dotenv.config({
//   path: process.env.TEST_ENV ? `./env-files/.env.${process.env.TEST_ENV}` :
//   `./env-files/.env.dev`
// })
dotenv.config({
  path: path.resolve('testdata', '.env')//`./testdata/.env`
})
/**
 * See https://playwright.dev/docs/test-configuration.
 */
export default defineConfig({
```

```

//grep:/UI/,
//greplnvert:/sanity/,
//globalSetup: "./global-setup.ts",
testDir: './',
// expect:{
//   toHaveScreenshot:{
//     maxDiffPixels:20,
//     //maxDiffPixelRatio:0.1
//   }
// },
/* Run tests in files in parallel */
fullyParallel: false,
/* Fail the build on CI if you accidentally left test.only in the source code. */
forbidOnly: !!process.env.CI,
/* Retry on CI only */
retries: process.env.CI ? 1 : 0,
/* Opt out of parallel tests on CI. */
workers: process.env.CI ? 1 : 1,
/* Reporter to use. See https://playwright.dev/docs/test-reporters */
//reporter: [ 'html',{open:'on-failure'}],
//reporter : [['junit',{outputFile:'xyz/report4junit.xml'}]],
//reporter:[["html"],["github"]],
//reporter:process.env.CI?'github':'html',
//reporter:"allure-playwright",
//reporter:[["allure-playwright",
{outputFolder:"GoogleAllureResults4"}],["html"]],
/* Shared settings for all the projects below. See
https://playwright.dev/docs/api/class-testoptions. */
use: {
  /* Base URL to use in actions like `await page.goto('/')`. */
  // baseURL: 'http://127.0.0.1:3000',
  baseURL: "https://restful-booker.herokuapp.com",
  extraHTTPHeaders: {
    Accept: "application/json",
    "Content-Type": "application/json",
    // Authorization: "Basic YWRtaW46cGFzc3dvcmQxMjM="
  },
  /* Collect trace when retrying the failed test. See
https://playwright.dev/docs/trace-viewer */
  screenshot: 'only-on-failure',
  video: 'retain-on-failure',
  // trace:'on',
  //storageState:"./playwright/.auth/auth.json",
},
timeout: 30000,
/* Configure projects for major browsers */

```

```

projects: [
  // {
  //   name: "setup",
  //   testMatch: /\.*\setup\.ts/,
  //   //testMatch: "global.setup.ts"
  // },
  {
    name: 'chromium',
    //dependencies:["setup"],
    use: {
      ...devices['Desktop Chrome'],
      //storageState: "../playwright/.auth/auth.json"
    }
  },
  // {
  //   name: 'firefox',
  //   //dependencies:["setup"],
  //   use: {
  //     ...devices['Desktop Firefox'],
  //     //storageState: "../playwright/.auth/auth.json"
  //   },
  // },
  // {
  //   name: 'webkit',
  //   use: { ...devices['Desktop Safari'] },
  // },

  // {
  //   name:'iphone14MaxPro',
  //   use:{ ...devices['iPhone 14 Pro Max']}
  // },
  /* Test against mobile viewports. */
  // {
  //   name: 'Mobile Chrome',
  //   use: { ...devices['Pixel 5'] },
  // },
  // {
  //   name: 'Mobile Safari',
  //   use: { ...devices['iPhone 12'] },
  // },

  /* Test against branded browsers. */
  // {
  //   name: 'Microsoft Edge',
  //   use: { ...devices['Desktop Edge'], channel: 'msedge' },
  // },

```



```

// {
//   name: 'Google Chrome',
//   use: { ...devices['Desktop Chrome'], channel: 'chrome' },
// },
],

/* Run your local dev server before starting the tests */
// webServer: {
//   command: 'npm run start',
//   url: 'http://127.0.0.1:3000',
//   reuseExistingServer: !process.env.CI,
// },
});

```

=====

[In Built Reporters | List, Line, Dot, HTML, Blob, Json, Junit, Github Reporters | Playwright Tutorials](#)

Playwright Test comes with a few built-in reporters for different needs and ability to provide custom reporters. All built-in reporters show detailed information about failures, and mostly differ in verbosity for successful runs.

List Reporter - List reporter is default reporter (except on CI where the dot reporter is default). It prints a line for each test being run.

Line Reporters - Line reporter is more concise than the list reporter. It uses a single line to report the last finished test, and prints failures when they occur. Line reporter is useful for large test suites where it shows the progress but does not spam the output by listing all the tests.

Dot Reporter - Dot reporter is very concise - it only produces a single character per successful test run. List reporter is default (except on CI where the dot reporter is default).

Character	Description
•	Passed
F	Failed
x	Failed or timed out - and will be retried
±	Passed on retry (flaky)
T	Timed out
◦	Skipped

HTML Reporter - HTML reporter produces a self-contained folder that contains report for the test run that can be served as a web page.

Blob reporter - Blob reports contain all the details about the test run and can be used later to produce any other report.

JSON reporter - JSON reporter produces an object with all information about the test run. Most likely you want to write the JSON to a file. When running with `--reporter=json`, use `PLAYWRIGHT_JSON_OUTPUT_NAME` environment variable.

JUnit reporter - JUnit reporter produces a JUnit-style xml report. Most likely you want to write the report to an xml file. When running with `--reporter=junit`, use `PLAYWRIGHT_JUNIT_OUTPUT_NAME` environment variable:

GitHub Actions annotations - You can use the built in github reporter to get automatic failure annotations when running in GitHub actions.

Reporters on CI - You can use different reporters locally and on CI. For example, using concise 'dot' reporter avoids too much output. This is the default on CI.

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  // Concise 'dot' for CI, default 'list' when running locally
  reporter: process.env.CI ? 'dot' : 'list',
});
```

Playwright Built-in Reporters Explained

Reporters are simply the way Playwright shows you the results of your tests after they run. Think of them as different display styles for the same information.



Layman's Terms For Each Reporter

List Reporter (Default)

- ✓ Test 1 - Login works
- ✓ Test 2 - Booking works
- ✗ Test 3 - Delete fails
- ✓ Test 4 - Search works

- Prints one line per test as it runs

- Like a shopping receipt — every item listed one by one
- Default on your local machine
- Good when you have a small number of tests

— Line Reporter

X Test 3 - Delete fails ← Only shows the LAST test + any failures

- Only shows one line at a time — updates as each test finishes
- Like a scoreboard that keeps updating — doesn't show the full history
- Good for large test suites — doesn't flood your screen with 500 lines
- Still shows failures when they happen

• Dot Reporter (Default on CI)

... X

- Just prints a single dot for each passing test
- An X or F for failing tests
- Like a progress bar made of dots
- Very minimal — used on CI/CD pipelines (automated servers) where screen space matters
- Default when running on CI (like GitHub Actions, Jenkins)

All Reporters At A Glance

Reporter	Output Style	Best For	Default?
List	One line per test	Small test suites, local dev	 Local machine
Line	Single line updating	Large test suites	
Dot	One dot per test	CI/CD pipelines, servers	 CI environment
HTML	Full web page report	Sharing results with team	
JSON	Raw JSON data	Feeding results to other tools	

JUnit	XML format	Java-based CI tools	✗
Blob	Binary format	Merging reports from multiple machines	✗
Github	Github-flavored output	Github Actions specifically	✗

How To Use Them

In `playwright.config.ts`:

```
export default defineConfig({
  reporter: "list", // or "line", "dot", "html", "json"
})
```

From the terminal:

```
npx playwright test --reporter=list
npx playwright test --reporter=dot
npx playwright test --reporter=html
```

Multiple reporters at once:

```
reporter: [
  ["list"],          // Show in terminal
  ["html"],          // Also generate HTML report
  ["json", { outputFile: "results.json" }] // Also save JSON
]
```

Real World Analogy

Think of it like different ways a teacher can mark attendance:

- **List Reporter** — Teacher calls out every student's name one by one — you hear every single one (*good for small class*)
- **Line Reporter** — Teacher only announces the last student checked and mentions anyone absent — less noise (*good for big class*)
- **Dot Reporter** — Teacher just makes a tick mark for each student — very fast, minimal info (*good for huge class*)
- **HTML Reporter** — Teacher produces a full printed report card with colors, charts and details you can share with parents

- **JSON Reporter** — Teacher gives raw data to the school's computer system to process further

🔑 The key point — all reporters show the same test results, they just display them differently. Choose based on your needs: detailed feedback locally, minimal output on servers, pretty reports for sharing with your team!

In VSCode:

- In the tests folder create 2 files **reporter1practice.spec.ts** and **reporter2.spec.ts** and include the following:

```
import {test, expect} from "@playwright/test";

test("Reporter Practice 1", async({page})=>{
  await page.goto("https://www.google.com");
  await expect(page).toHaveTitle("Google");
})

test("Reporter Practice 2", async({page})=>{
  await page.goto("https://www.google.com");
  await expect(page).toHaveTitle("Google");
})

test("Reporter Practice 3", async({page})=>{
  await page.goto("https://www.google.com");
  await expect(page).toHaveTitle("Google");
})
```

(reporter1.spec.ts)

```
import {test, expect} from "@playwright/test";

test("Reporter Practice 4", async({page})=>{
  await page.goto("https://www.google.com");
  await expect(page).toHaveTitle("Google");
})

test("Reporter Practice 5", async({page})=>{
  await page.goto("https://www.google.com");
  await expect(page).toHaveTitle("Google");
})
```

```
test("Reporter Practice 6", async({page})=>{
  await page.goto("https://www.google.com");
  await expect(page).toHaveTitle("Google");
})
```

(reporter2practice.spec.ts)

- Use the below commands and configurations in the playwright.config.ts file for displaying the reporters in the terminal accordingly

Here's a comprehensive overview of the multiple ways to configure and use reporters in Playwright:

Playwright Reporter Commands & Configuration

1. Via CLI (**--reporter** flag)

Single reporter

npx playwright test --reporter=list

npx playwright test --reporter=dot

npx playwright test --reporter=line

npx playwright test --reporter=html

npx playwright test --reporter=json

npx playwright test --reporter=junit

Multiple reporters

npx playwright test --reporter=dot,html

With output file

npx playwright test --reporter=json --output=results.json

2. Via **playwright.config.ts**

// Single reporter

reporter: 'html'

// Reporter with options

reporter: [['html', { open: 'never', outputFolder: 'my-report' }]]

```
// Multiple reporters
reporter: [
  ['list'],
  ['html', { outputFolder: 'html-report' }],
  ['json', { outputFile: 'results.json' }],
  ['junit', { outputFile: 'junit.xml' }],
]
```

3. Built-in Reporters

Reporter	Command	Description
List	<code>--reporter=list</code>	Verbose line-by-line output (default in CI)
Dot	<code>--reporter=dot</code>	Minimal dot-per-test output
Line	<code>--reporter=line</code>	Compact single-line progress
HTML	<code>--reporter=html</code>	Interactive HTML report
JSON	<code>--reporter=json</code>	Machine-readable JSON output
JUnit	<code>--reporter=junit</code>	XML for CI tools (Jenkins, etc.)
GitHub	<code>--reporter=github</code>	Annotations for GitHub Actions
Blob	<code>--reporter=blob</code>	For merging sharded test runs

4. Open/Show HTML Report

Auto-opens after test run (default on failure)
`npx playwright show-report`

Specify custom folder
`npx playwright show-report my-report`

5. Merge Reports (for Sharded Runs)

Merge blob reports from shards

```
npx playwright merge-reports --reporter html ./blob-reports
```

```
npx playwright merge-reports --reporter json ./blob-reports > merged.json
```

6. Custom Reporter (class-based)

```
// custom-reporter.ts
```

```
class MyReporter {  
  onBegin(config, suite) { ... }  
  onTestBegin(test) { ... }  
  onTestEnd(test, result) { ... }  
  onEnd(result) { ... }  
}
```

```
// In config:
```

```
reporter: [['./custom-reporter.ts']]
```

7. Environment Variable Override

Override reporter without changing config

```
PLAYWRIGHT_HTML_REPORT=my-folder npx playwright test --reporter=html
```

Tip: You can combine reporters so tests print to the terminal *and* generate an HTML report simultaneously:

```
npx playwright test --reporter=list,html
```

Which Terminal to Use?

All Playwright commands work in all three terminals — PowerShell, Bash, and CMD. But here's the practical guide:

Terminal		When to Use		
PowerShell	Windows	development	(recommended	for
I	Windows)			
Bash	Mac, Linux, or Git Bash on Windows			

CMD Works but least recommended — older, limited features

Simple rule: Use whatever terminal your project normally uses. If you're on Windows → PowerShell. If you're on Mac/Linux → Bash. All the **npx playwright test** commands are identical across all three.

What is Sharding? (In Layman's Terms)

Imagine you have 1000 tests to run. Running them one by one takes 60 minutes.

Sharding = splitting the work across multiple workers/machines so they all run at the same time and finish much faster.

Real-world analogy 🍕

You need to deliver 100 pizzas. Instead of one driver doing all 100 trips, you split it across 4 drivers — each delivers 25 pizzas simultaneously. The whole job finishes 4x faster.

That's exactly what sharding does with your tests.

How it works in Playwright

Machine 1 — runs first quarter of tests

npx playwright test --shard=1/4

Machine 2 — runs second quarter

npx playwright test --shard=2/4

Machine 3 — runs third quarter

npx playwright test --shard=3/4

Machine 4 — runs fourth quarter

npx playwright test --shard=4/4

Each machine runs independently and simultaneously. When all 4 finish, you merge their reports into one:

npx playwright merge-reports --reporter html ./blob-reports

When should YOU use sharding?

Situation	Use Sharding?
Small project, < 100 tests	✗ Not needed
Large project, 500+ tests	✓ Yes
Running in CI/CD (GitHub Actions, Jenkins)	✓ Yes
Running on your local laptop	✗ Usually not needed
Tests take more than 10 mins	✓ Yes

Quick Summary

- PowerShell/Bash/CMD → all work, pick based on your OS
- Sharding → splitting your tests across multiple machines so they run *at the same time* instead of one after another, saving you lots of time on large projects

Playwright Reporter Commands — Windows Syntax

✓ PowerShell

Single reporter

```
npx playwright test --reporter=list  
npx playwright test --reporter=html  
npx playwright test --reporter=json
```

Multiple reporters

```
npx playwright test --reporter=list,html
```

With output file

```
npx playwright test --reporter=json --output=results.json
```

Open HTML report

```
npx playwright show-report
```

Sharding

```
npx playwright test --shard=1/4
```

✅ CMD (Command Prompt)

:: Single reporter

`npx playwright test --reporter=list`

`npx playwright test --reporter=html`

`npx playwright test --reporter=json`

:: Multiple reporters

`npx playwright test --reporter=list,html`

:: With output file

`npx playwright test --reporter=json --output=results.json`

:: Open HTML report

`npx playwright show-report`

:: Sharding

`npx playwright test --shard=1/4`

Note: Comments in CMD use `::` instead of `#`

✅ Bash (Git Bash on Windows)

Single reporter

`npx playwright test --reporter=list`

`npx playwright test --reporter=html`

`npx playwright test --reporter=json`

Multiple reporters

`npx playwright test --reporter=list,html`

With output file

`npx playwright test --reporter=json --output=results.json`

Open HTML report

`npx playwright show-report`

Sharding

`npx playwright test --shard=1/4`

Key Differences Between the Three

Feature	PowerShell	CMD	Bash
Comment symbol	#	::	#
The npx commands	Identical	Identical	Identical
Environment variables	\$env:VAR="value"	set VAR=value	export VAR=value
Multi-line commands	Use ` backtick	Use ^ caret	Use \ backslash

Setting Environment Variable (the one difference that matters)

If you ever need to set a custom report folder:

PowerShell

```
$env:PLAYWRIGHT_HTML_REPORT="my-folder"; npx playwright test --reporter=html
```

:: CMD

```
set PLAYWRIGHT_HTML_REPORT=my-folder && npx playwright test --reporter=html
```

Bash

```
PLAYWRIGHT_HTML_REPORT=my-folder npx playwright test --reporter=html
```

Bottom line: The **npx playwright test** commands are 100% identical across all three terminals on Windows. The only real differences show up with comments and environment variables.

Allure Report in Playwright with Screenshot, Videos, Trace Viewer and Retries | Playwright Tutorials

By integrating Allure with Playwright, users can leverage Allure's powerful reporting capabilities to enhance the visibility and analysis of their test results.

1 - Installation of allure-playwright and allure-commandline package -

```
npm install allure-playwright
```

```
npm install -g allure-commandline --save-dev
```

(Forgot to mention this command in Video - Don't forget to install it)

2 - Generate Allure Result files -

Generate Allure Result files through command line -

Generate result files in default allure-results folder -

```
npx playwright test --reporter=allure-playwright
```

Generate result files in specified folder -

CMD or command prompt (for Windows) -

```
set ALLURE_RESULTS_DIR=ResultFolderName
```

```
npx playwright test --reporter=allure-playwright
```

Powershell Terminal (for Windows) -

```
$env:ALLURE_RESULTS_DIR="ResultFolderName"; npx playwright test --reporter=allure-playwright
```

Bash Terminal (for Mac / Linux) -

```
ALLURE_RESULTS_DIR=ResultFolderName npx playwright test --reporter=allure-playwright
```

Generate Allure Result files through Playwright.config.ts file -

Generate result files in default allure-results folder -

```
reporter:"allure-playwright"
```

Generate result files in specified folder -

```
Reporter:[["allure-playwright", {outputFolder : ResultFolderName}]]
```

3 - Generate Allure Report files -

Generate Allure Report files-

Generate Report files in default allure-report folder

```
allure generate ResultFolderName --clean
```

If want to generate Allure Report files as Single HTML file -

```
allure generate allure-results --clean --single-file
```

Generate Report files in specified folder -

```
allure generate ResultFolderName -o ReportFolderName --clean
```

Note - To Overwrite the old report files it's better to add --clean at the end , when you are generating report files from results files.

4 - Open Allure Report from Report Files -

allure open ReportFolderName

Alternate Step - Generate Allure Report files and Open Report in single command (Skip Step 3 and 4)

allure serve ResultFolderName

Other Points -

- We need to set values for screenshot, video, trace keys in the playwright config file to show it with allure report.
- On Adding retries values more than zero in playwright config file, retries execution details will appear with allure report.

If you Just Interested in generating Allure Report . Just provide below command -

- 1 - Install allure-playwright (Require Only first time for setup) - `npm install allure-playwright`
- 2 - Install allure-commandline - `npm install -g allure-commandline --save-dev`
- 3 - Generate Allure Results files - `npx playwright test --reporter=allure-playwright`
- 4 - Generate Allure Report files and Open Report - `allure serve allure-results`

Refer to this website for installation:

<https://www.npmjs.com/package/allure-playwright>

In VSCode for practicing generating the allure report use the code from reporterpractice1.spec.ts and reporterpractice2.spec.ts

Push Code to Github | Clone Repository | Playwright Tutorials

GitHub is a web-based platform that provides hosting for Git repositories, Git provides a robust framework for managing code changes, facilitating collaboration, and maintaining a reliable history of your project's development.

1. **Version Control:** Git keeps track of every change made to the codebase, allowing you to view historical changes and revert to previous versions.
 2. **Collaboration:** Git allows multiple developers to work on the same project simultaneously.
 3. **Branching and Merging:** Git enables branching, which allows you to work on new features or fixes in isolated environments. Once the work is complete, these branches can be merged back into the main codebase.
 4. **Backup and Recovery:** Storing code in Git provides a backup of your work. If a local machine fails or data is lost, you can recover the code from the remote Git repository.
 5. **Code Review:** Git supports code review workflows where changes can be reviewed and approved before being integrated into the main project. This process improves code quality and ensures standards are met.
 6. **Audit Trail:** Git maintains a detailed history of changes, including who made each change and why.
 7. **Distributed Development:** Git is a distributed version control system, meaning every developer has a full copy of the code repository. This decentralization enhances redundancy and ensures that development can continue even if the central repository is unavailable.
- **GitHub HTTPS** refers to using the **Hypertext Transfer Protocol Secure (HTTPS)** to interact with your GitHub repositories.
 - **GitHub SSH** is a method of securely connecting to your GitHub account using the **Secure Shell Protocol (SSH)**. Instead of relying on usernames and passwords, SSH uses a pair of cryptographic keys.

Git Commands to Push Code -

git init

- This initializes a new Git repository in the current directory. It creates a hidden .git directory where all the Git configuration and repository data will be stored.

git add

- This stages all the changes (new files, modified files, deleted files) in the current directory and its subdirectories for the next commit. The . is a shorthand for "the current directory".

Or

git add <filename>

- This stages the specified file (<filename>) for the next commit. Replace <filename> with the actual name of the file you want to add. If you want to add all files, you can use git add .

git commit -m "any comment"

- This creates a new commit with the staged files. The -m option allows you to specify a commit message inline.

Note -

For the first time git User you can see the message "Author identity unknown" on committing the code for the first time .

You can set your identity (Email and Name) by providing below commands -

```
git config --global user.email "you@example.com"
```

```
git config --global user.name "Your Name"
```

git branch -M main

- This command renames the current branch to `main`. The `-M` flag forces the rename if a branch with the new name already exists.

```
git remote add origin https://github.com/Wish-Infinite/PracticeRepository.git
```

or

```
git remote add origin git@github.com:Wish-Infinite/PracticeGit4.git
```

- This adds a new remote repository named origin. The URL provided is the location of the remote repository on GitHub. This allows you to push and pull changes to and from the remote repository.

git push -u origin main

- This pushes the commits from your local main branch to the main branch in the remote repository (origin). The -u flag sets origin/main as the default upstream branch, so future git push commands can be run without specifying the remote and branch.

git pull origin main

- This is used to fetch and integrate changes from a remote repository into your local branch.

Clone Repository -

Git clone <repository URL>

Refer to this video on how to upload a folder/project to github:

<https://www.youtube.com/watch?v=p8bZBvcFPuk>

What is a Pull Request?

In GitHub, a pull request (often known as PR) is a way to propose changes to the codebase. It's like saying, "Hey, I've made these changes, can you please review them and, if everything looks good, merge them into the main project / branch?"

How Does a Pull Request Work?

1. **Create a Branch:** You start by creating a new branch, which is basically a separate version of the project where you make your changes. This allows you to work on your updates without affecting the main project.
2. **Make Changes:** You then make your changes on this branch. It could be fixing a bug, adding a new feature, or improving documentation.
3. **Add & Commit Your Changes:** Once you're happy with your changes, you commit them. A commit is like saving your work with a note explaining what you did.
4. **Push to GitHub:** Next, you push your changes from your local machine to GitHub. This means you upload your branch with the changes to the repository on GitHub.
5. **Create a Pull Request:** Now, you go to GitHub and create a pull request. You describe what changes you've made, why they're necessary, and ask for feedback or approval to merge these changes into the main branch.
6. **Review Process:** Others on your team will review the pull request. They might leave comments, suggest changes, or approve it if everything looks good.
7. **Merge the Pull Request:** Once everyone is happy with the changes, the pull request is merged. This means your changes are now part of the main project.

- `git branch branchName`
- `git checkout branchName`

Or

- `git checkout -b branchName`
- `git add.`
- `git commit -m "any comment"`
- `git push origin branchName`

What is a Pull Request?

Layman's Terms First

Imagine you are working on a shared Google Doc with your team.

Instead of directly editing the main document, you make your own copy, make changes, then raise your hand and say "Hey team, I made some changes — can someone check them and approve before we add to the main doc?"

That "raising your hand and asking for approval" is exactly a Pull Request (PR).

Real World Analogy

Think of it like a company approval process:

You write a report (make code changes)



You send it to your manager for review (open a Pull Request)



Manager reads it, suggests edits (teammate reviews your code)



You make the suggested changes (update your code)



Manager approves and files it officially (PR gets merged into main code)

How it Works Step by Step

Step 1 — You have the main project

main branch ← this is the "official" working code

Step 2 — You create your own branch (your own copy to work on)

git checkout -b my-new-feature

main branch (untouched, safe)



my-new-feature (your personal working copy)

Step 3 — You make your changes and push them

git add .

git commit -m "added login feature"

git push origin my-new-feature

Step 4 — You open a Pull Request on GitHub

You go to GitHub and click "Compare & Pull Request"

You are essentially saying:

"I've finished my changes on my branch — please review and merge them into the main branch"

Step 5 — Teammates Review

Your teammates can:

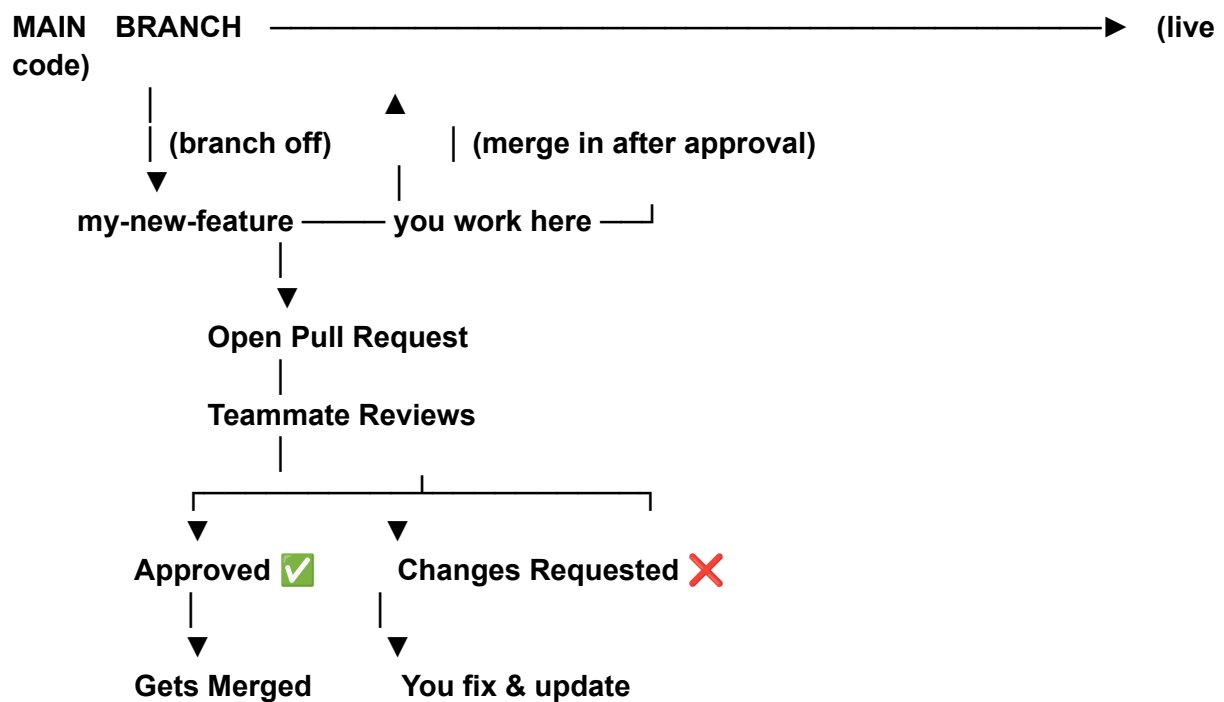
-  Approve — looks good, merge it
-  Comment — ask questions or suggest improvements
-  Request changes — something needs fixing before merging

Step 6 — Gets Merged

Once approved, the code is merged into main — your changes become part of the official project.

my-new-feature →→→→→ merged into →→→→→ main branch

Visual Flow



Why Not Just Edit the Main Code Directly?

Direct Edit to Main	Using a Pull Request
Risky — one mistake breaks everything	Safe — changes reviewed before merging
No one checks your work	Team reviews your code

Hard to track what changed and why	Clear history of what changed and why
No way to discuss changes	Can comment, discuss, suggest

Key Terms Simply Explained

Term	What it means in plain English
Branch	Your own personal copy of the project to work on
Commit	Saving a snapshot of your changes
Push	Uploading your changes to GitHub
Pull Request	Asking the team to review and accept your changes
Merge	Officially adding your changes into the main code
Review	A teammate checking your changes before approval
Approve	Teammate gives the green light
Request Changes	Teammate says "fix this first"

One Line Summary

A Pull Request is simply a formal way of saying "I made changes, please check them and add them to the main project" — it keeps the main code safe, lets teams collaborate, and makes sure nothing broken gets in without someone checking it first.

What is CI/CD?

- **CI (Continuous Integration):** "It's the practice of automating the integration of code changes from multiple contributors into a **project / codebase**."
- **CD (Continuous Delivery/Deployment):** "This takes it a step further by automating the delivery of the integrated code to production."

Steps -

- Create Yaml File -
 - Install / Reinstall Playwright using common `npm init playwright@latest`
 - Install / Reinstall Playwright using Playwright VS Code extension
 - Manually Create Yaml file at `.github/workflows` location in project folder
- Playwright.config.ts file configurations.
- Job execution on Github Actions when user Push Code / Raise PR / Merge Code
- Report Generation

CI/CD with GitHub Actions & Playwright — Full Explanation

What is CI/CD? (Layman Terms First)

CI — Continuous Integration

Analogy: Group assignment at school

Imagine 5 students working on the same project. Instead of everyone working separately for weeks and combining everything at the last minute (causing chaos), they merge their work together every single day and check if everything still works fine.

That daily merging and checking = Continuous Integration

In real terms:

- Multiple developers push code changes regularly
- Every time someone pushes, the system automatically checks if the code still works
- Problems are caught early instead of at the last minute

CD — Continuous Delivery/Deployment

Analogy: Amazon warehouse

Once a package is packed and quality-checked, it automatically gets shipped to the customer. No one manually has to say "okay ship this now" — it just happens automatically.

That automatic shipping = Continuous Deployment

In real terms:

- Once code passes all checks, it automatically gets delivered to the live application
 - No manual steps needed to release
-

CI/CD Together in One Picture

Developer writes code



Pushes to GitHub



CI kicks in automatically ←—— This is the magic moment



Runs all Playwright tests



✅ All pass → CD deploys to live app

❌ Any fail → Team gets notified, nothing broken ships

GitHub Actions — What is it?

Think of GitHub Actions as a robot sitting inside GitHub that watches your project 24/7 and automatically does tasks whenever something happens (like someone pushing code or raising a PR).

You give this robot a set of instructions written in a YAML file — and it follows them every time the trigger happens.

The Steps Explained

Step 1 — Create YAML File

A YAML file is basically the instruction manual for the robot (GitHub Actions).

It tells GitHub:

- *When* to run (on push? on PR? on merge?)
- *What* to run (install dependencies, run Playwright tests, generate report)

3 ways to create it:

Option A — Using npm command

`npm init playwright@latest`

This automatically creates the YAML file for you along with the Playwright setup.

Option B — Using VS Code Playwright Extension

The extension has a GUI that generates the YAML file without writing any code manually.

Option C — Manually create the file

You create the file yourself at this exact location in your project:

```
your-project/
├── .github/
│   └── workflows/
│       └── playwright.yml ← create this file here
├── tests/
└── playwright.config.ts
```

A basic YAML file looks like this:

name: Playwright Tests

on:

push: # run when code is pushed

pull_request: # run when PR is raised

jobs:

test:

runs-on: ubuntu-latest

steps:

- **uses:** actions/checkout@v3

- **uses:** actions/setup-node@v3

- **run:** npm install

- **run:** npx playwright install

- **run:** npx playwright test

Step 2 — playwright.config.ts Configurations ⚙️

This is the settings file for Playwright. It tells Playwright how to behave during the CI run.

Important settings for CI:

```
export default defineConfig({
  // Don't open browser visually in CI (no screen available)
  use: {
    headless: true,
  },

  // Which reporter to use
  reporter: [['html'], ['junit']],

  // How many workers (parallel runs)
  workers: 4,
});
```

Step 3 — Job Execution Triggers 🛎️

The robot (GitHub Actions) runs your Playwright tests automatically on 3 events:

Trigger	What it means in plain English
Push Code	Someone uploads new code to GitHub
Raise PR	Someone asks for their code to be reviewed and merged
Merge Code	Approved code gets added to the main branch

Any of these 3 happen

↓
GitHub Actions robot wakes up
↓
Runs all Playwright tests automatically
↓
Pass ✅ or Fail ❌

Step 4 — Report Generation 📊

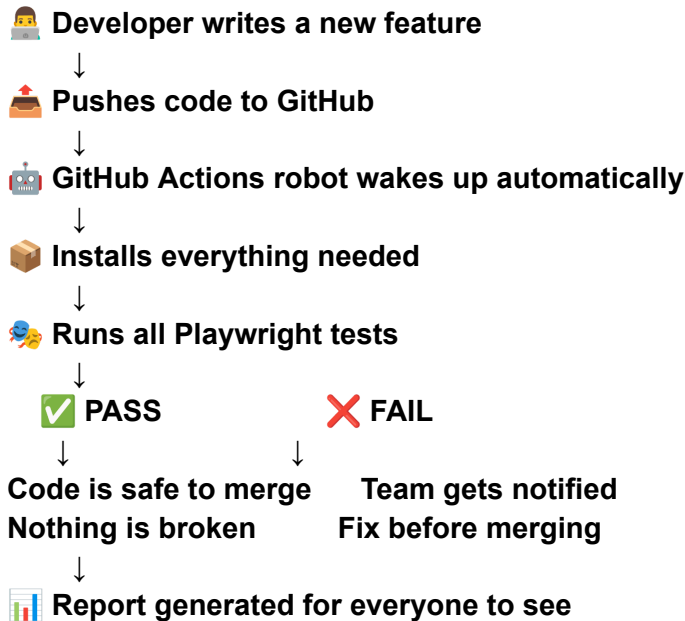
After tests run, GitHub Actions automatically generates a report so you can see:

- Which tests passed ✅
- Which tests failed ❌

- Screenshots of failures
- How long tests took

This report is uploaded and visible directly on GitHub — no need to run anything locally.

The Full Big Picture (Layman)



One Line Summary of Everything

CI/CD with GitHub Actions means your Playwright tests run automatically every time anyone touches the code — so broken code never sneaks into the project without anyone noticing.

CI using Jenkins | Jenkins Setup & Integration with Local Playwright Project | Playwright Tutorials

What is CI/CD?

- **CI (Continuous Integration):** "It's the practice of automating the integration of code changes from multiple contributors into a **project / codebase.**"
- **CD (Continuous Delivery/Deployment):** "This takes it a step further by automating the delivery of the integrated code to production."

Jenkins can be integrated with Playwright with 2 ways .

- Without using pipeline
- By using Pipeline (Jenkins supports Docker agents for pipelines.)

Important Steps -

- Install Prerequisites - **NodeJS, Java , Jenkins**
- Create Test Script and Configure playwright.config.ts
- Create Jenkins Job.
- Provide Build steps
- Configure HTML Report with Jenkins
- Execute build Manually
- Verify Generated report and log.
- Trigger build periodically
- Parameterize Jenkins job (If Required)

To Configure HTML report - `System.setProperty("hudson.model.DirectoryBrowserSupport.CSP", "")`

Note - If you want to use **Jenkins.war** file rather than installing Jenkins using **jenkins.exe**, then you can use the **java -jar jenkins.war --httpPort=8080** command to run the file. (As explained in Video)

CI using Jenkins | Jenkins Integration with Playwright Git Repository | Ngrok | Playwright Tutorials

Webhooks: "A webhook is a way for an application, like GitHub, to send real-time updates to another application, such as Jenkins. For example, when a developer pushes code to GitHub, a webhook can trigger a Jenkins job automatically."

SCM Polling: "Polling SCM stands for 'Source Code Management' polling. Instead of waiting for a webhook, Jenkins periodically checks (or polls) GitHub to see if any code changes were made and then triggers a job if there's a new commit."

ngrok: "Since Jenkins might be running locally, ngrok allows us to expose the local Jenkins server to the internet so that GitHub can communicate with it through the webhook."

NGROK Setup Steps-

- Download Ngrok for Windows.
- Extract ngrok at any location.
- Open CMD at the same location.
- `ngrok config add-authtoken <auth token>` (Signup on ngrok website to get token)
- `ngrok http http://localhost:8080`

Refer to this video on the configurations to specify on Jenkins:

https://www.youtube.com/watch?v=fjph_JWx0as&list=PL83cimSRP5ZmwhC6u255huRwSi9tIP-nc&index=61